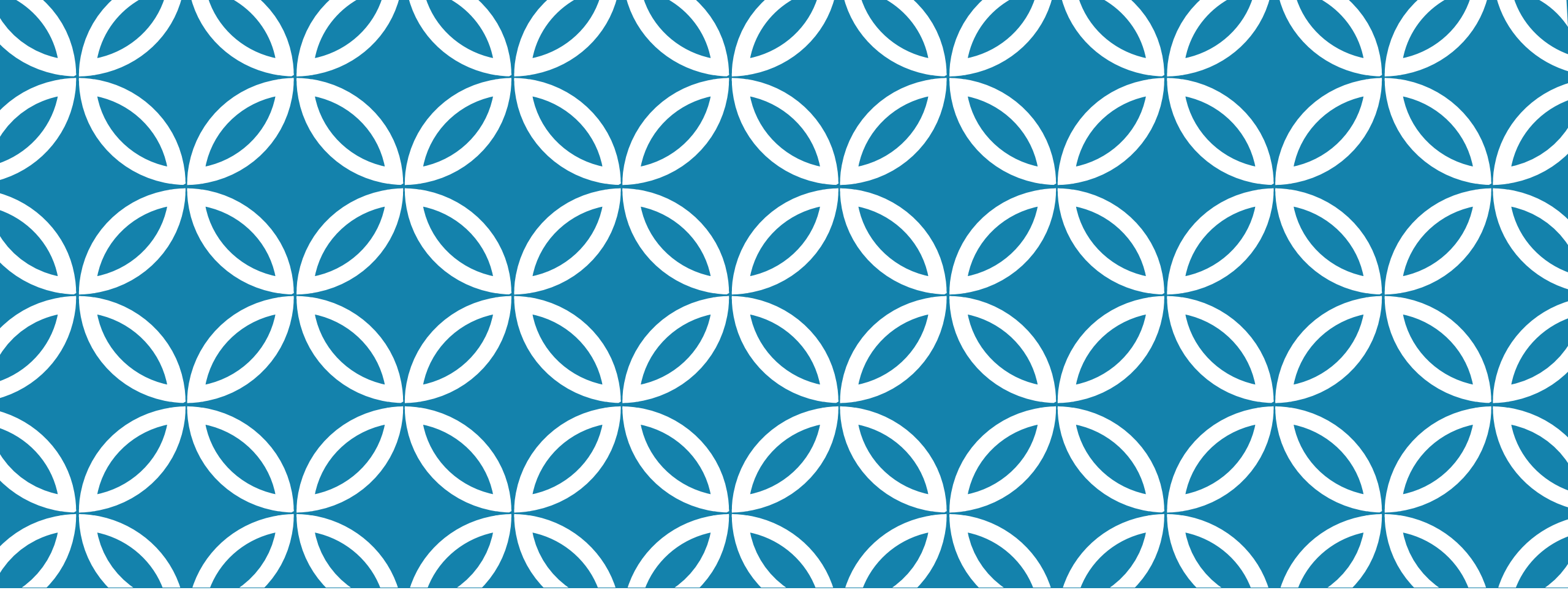


NESNE TABANLI PROGRAMLAMA-I

Öğr. Gör. Erhan AKAGÜNDÜZ

INHERITANCE NEDİR?

- Büyük yazılım projeleri genellikle birden fazla modülden oluşur.
- Bu modüller birçok ortak özelliğe ve işlevlere sahiptir.
- Eğer her alt modül için aynı özellikleri yeniden tanımlamak zorunda kalırsak, bu büyük bir iş yükü oluşturur.
- İşte bu noktada Inheritance (miras alma) kavramı devreye girer.
- Miras alma, bir sınıfın özelliklerini ve işlevlerini başka sınıflara aktarmamızı sağlar.
- Böylece kod tekrarını önler ve yazılımın sürdürülebilirliğini artırır.

INHERITANCE'IN TÜRKÇE KARŞILIĞI

- Inheritance Türkçe'de kalıtım veya miras alma olarak adlandırılır.
- Nesne tabanlı programlamada (OOP) temel kavramlardan biridir.
- Normalde her nesne için ayrı bir sınıf oluşturur ve özelliklerini tanımlarız.
- Miras alma ile artık aynı özellikleri tekrar yazmaya gerek kalmaz.
- Kod tekrarının önlenmesi, projelerde büyük kolaylık sağlar.
- Aynı zamanda projeyi daha düzenli ve okunabilir hâle getirir.

GÜNLÜK HAYATTAN ÖRNEK

- Elimizde bir **Taşıt** sınıfı olduğunu düşünelim.
- Bu sınıftan türeyen alt sınıflar: **Araba** ve **Motosiklet**.
- Araba ve motosiklet, taşıtın tüm özelliklerini otomatik olarak alır.
- Alt sınıflar bu özellikleri tekrar yazmak zorunda değildir.
- Taşıt sınıfı bir şablon görevi görür, alt sınıflar bu şablondan miras alır.
- Bu örnek, inheritance kavramını somut bir şekilde gösterir.

INHERİTANCE AVANTAJLARI

- Kod tekrarını önler ve geliştirme süresini kısaltır.
- Kodun bakımı ve güncellenmesi daha kolay hale gelir.
- Yeni alt sınıflar eklemek pratik ve hızlıdır.
- Ana sınıfta yapılan değişiklikler otomatik olarak alt sınıflara yansır.
- Kodun okunabilirliği artar ve hata riski azalır.
- Ekip çalışmasında inheritance kullanımı, uyum ve verimliliği artırır.

OOP İLE TEKRARLI KOD ÖRNEĞİ

- Her sınıf için aynı kodları tekrar yazmak gerekir.
- Kod hem uzun hem de hataya açık olur.
- Bakımı zor ve projeyi büyütmek maliyetli olur.
- OOP'nin temel amacı burada tam olarak sağlanamıyor.
- Inheritance ile bu kod tekrarından kurtulabiliriz.
- Alt sınıflar ana sınıfın tüm özelliklerini devralabilir.

```
class Akademisyen:
    def __init__(self, adi, soyadi, numarasi):
        self.adi = adi
        self.soyadi = soyadi
        self.numarasi = numarasi
    def giris(self):
        print("Giriş Yapıldı")
    def cikis(self):
        print("Çıkış yapıldı")
```

KULLANICI SINIFI İLE INHERİTANCE - 1

```
class Kullanici:
    def __init__(self, adi, soyadi, numara):
        self.adi = adi
        self.soyadi = soyadi
        self.numara = numara
    def giris(self):
        print("Giriş Yapıldı")
    def cikis(self):
        print("Çıkış yapıldı")

class Akademisyen(Kullanici):
    pass
class Personel(Kullanici):
    pass
class Ogrenci(Kullanici):
    pass
```

KULLANICI SINIFI İLE INHERİTANCE - DEVAMI

- Alt sınıflar **Kullanici** sınıfından miras alıyor.
- Ortak özellik ve işlevleri tekrar yazmak gerekmedi.
- pass anahtar kelimesi, alt sınıfın yeni özellik eklemediğini gösterir.
- Kod daha kısa ve yönetilebilir oldu.
- Ana sınıfta yapılan değişiklikler alt sınıflara otomatik yansır.
- Bu yöntem büyük projelerde çok kullanışlıdır.

NESNE OLUŞTURMA VE KULLANMA

- Alt sınıflardan nesneler oluşturduk.
- Ana sınıfın tüm özelliklerini ve metotlarını kullanabiliyorlar.
- Kod tekrarına gerek kalmadı.
- Kod okunabilir ve modüler hâle geldi.
- Alt sınıflar bağımsız şekilde çalışabilir.
- Büyük projelerde benzer yaklaşım zamandan tasarruf sağlar.

```
akademisyen = Akademisyen("Mustafa","Kaya",1236521)
personel = Personel("Mehmet","Yıldız",1539527)
ogrenci = Ogrenci("Can","Demir",1436589)
```

ÇIKTI ÖRNEĞİ

- Alt sınıflar, ana sınıfın özelliklerini sorunsuz kullanıyor.
- Kod tekrarından kaçınıldı ve esneklik sağlandı.
- Ana sınıfın değişiklikleri otomatik olarak alt sınıflara yansır.
- Kod okunabilirliği yüksek.
- Yazılım geliştirme süreci hızlandı.
- Büyük projelerde kullanıcı tipleri kolayca genişletilebilir.

```
Akademisyen
Mustafa
Kaya
1236521
Personel
Mehmet
Yıldız
1539527
Öğrenci
Can
Demir
1436589
```

OVERRİDİNG NEDİR?

- Overriding, Türkçe’de “üstüne yazmak” anlamına gelir.
- Alt sınıf, miras aldığı bir metodu kendi ihtiyaçlarına göre değiştirebilir.
- Ana sınıfın metodunu bozmaz, sadece alt sınıfa özel davranış ekler.
- Overriding ile alt sınıfa özgü ek parametreler ekleyebiliriz.
- Bu yöntem, inheritance ile birlikte kullanıldığında esnekliği artırır.
- Büyük projelerde farklı kullanıcı davranışları için sıklıkla tercih edilir.

OVERRİDİNG ÖRNEĞİ

- Alt sınıf kendi __init__() metodunu tanımlıyor.
- Ana sınıfın __init__() metodu override edilmiş oluyor.
- Alt sınıf kendi ek parametrelerini tanımlayabilir.
- Ana sınıfın diğer metodlarını kullanmak hâlâ mümkün.
- Kod tekrarına gerek kalmıyor.
- Yazılım esnekliği ve özelleştirilebilirliği artıyor.

```
class Akademisyen(Kullanici):
    def __init__(self, adi, soyadi, numara, dogum_tarihi):
        print("Akademisyen sınıfı fonksiyonu")
        self.adi = adi
        self.soyadi = soyadi
        self.numara = numara
        self.dogum_tarihi = dogum_tarihi
```

SUPER() FONKSİYONU NEDİR?

- super() fonksiyonu, alt sınıfın üst sınıfın özelliklerini kullanmasını sağlar.
- Özellikle override durumunda ana sınıfın __init__() metodunu çağırmak için kullanılır.
- Böylece alt sınıf hem kendi ek özelliklerini hem de ana sınıfın özelliklerini alabilir.
- Kod tekrarını önler ve daha sürdürülebilir bir yapı sunar.
- Büyük projelerde miras zincirinde kritik bir rol oynar.
- Alt sınıf, ana sınıfın tüm fonksiyonlarına erişim sağlayabilir.

SUPER() KULLANIMI

- Ana sınıfın __init__() fonksiyonu çağrıldı.
- Alt sınıf kendi ek özelliklerini tanımlayabiliyor.
- Kod tekrarına gerek kalmadı ve daha modüler oldu.
- Hem ana sınıf hem alt sınıfın yapıcı metodları çalıştı.
- Kod okunabilirliği ve sürdürülebilirliği yükseldi.
- Büyük projelerde alt sınıflara yeni özellik eklemek kolaylaştı.

```python
class Akademisyen(Kullanici):
    def __init__(self, adi, soyadi, numara, dogum_tarihi):
        print("Akademisyen sınıfı fonksiyonu")
        super().__init__(adi, soyadi, numara)
        self.dogum_tarihi = dogum_tarihi
```

INHERİTANCE İLE BÜYÜK PROJELERDE KULLANIM

- Büyük projelerde sınıflar arası ilişki çok karmaşık olabilir.
- Inheritance ile ortak özellikler tek bir ana sınıfta toplanabilir.
- Alt sınıflar sadece kendilerine özgü özellikleri ekler.
- Kod okunabilirliği artar, bakım kolaylaşır.
- Yeni özellik eklemek, projeyi bozmaz.
- Ekip içinde kod paylaşımı ve işbirliği kolaylaşır.

ÖZET – INHERITANCE'IN ÖNEMİ

- Inheritance, kod tekrarını önler ve projeyi daha düzenli hâle getirir.
- Alt sınıflar ana sınıfın özelliklerini kullanabilir ve gerekirse override edebilir.
- super() fonksiyonu, ana sınıfın metodlarını kullanmayı kolaylaştırır.
- Kod esnekliği ve modülerliği artar.
- Büyük projelerde yazılım geliştirmeyi hızlandırır.
- Bu kavram, OOP'de profesyonel yazılım geliştirme için kritik bir araçtır.

SONUÇ VE ÖNERİLER

- Python’da inheritance, sürdürülebilir ve okunabilir kod için vazgeçilmezdir.
- Kod tekrarını önler ve projeyi daha hızlı geliştirmenizi sağlar.
- Overriding ve super() ile alt sınıflara özel davranışlar eklenebilir.
- Büyük projelerde bu yapı, yazılımın ölçeklenebilirliğini sağlar.
- Kodun bakımı ve güncellenmesi daha kolay hâle gelir.
- Öğrendiğiniz bu konsepti günlük projelerinizde mutlaka uygulayın.